

# Package ‘mgcvUI’

March 26, 2026

**Title** Interactive GAM Builder for Real Estate Appraisers

**Version** 0.1.0

**Description** A Shiny application for building, diagnosing, and reporting Generalized Additive Models (GAMs) via 'mgcv'. Designed for real estate appraisers who need transparent, defensible nonlinear models. Optionally imports knot structure from 'earth' MARS models (e.g. via 'earthUI') to bridge exploratory and confirmatory modelling stages.

**License** AGPL (>= 3)

**URL** <https://github.com/wcraytor/mgcvUI>

**BugReports** <https://github.com/wcraytor/mgcvUI/issues>

**Encoding** UTF-8

**Roxygen** list(markdown = TRUE)

**RoxygenNote** 7.3.3

**Imports** shiny (>= 1.7.0), mgcv (>= 1.8-40), readxl, DT, ggplot2, officer, rmarkdown, broom, gratia, plotly, shinyFiles, rlang, htmltools, graphics, stats, tools, utils

**Suggests** earth (>= 5.3.0), DBI, RSQLite, jsonlite, bslib, callr, openxlsx, writexl, testthat (>= 3.0.0), tibble, withr, knitr, sysfonts, showtext, thematic

**Config/testthat/edition** 3

**VignetteBuilder** knitr

**NeedsCompilation** no

**Author** mgcvUI Developers [aut, cre]

**Maintainer** mgcvUI Developers <dev@example.com>

**Depends** R (>= 4.1.0)

## Contents

|                                   |   |
|-----------------------------------|---|
| mgcvUI-package . . . . .          | 2 |
| build_gam_formula . . . . .       | 3 |
| build_gam_knots . . . . .         | 4 |
| check_sign_consistency . . . . .  | 4 |
| detect_column_types . . . . .     | 5 |
| export_functions_sqlite . . . . . | 6 |

|                                    |           |
|------------------------------------|-----------|
| export_functions_zip . . . . .     | 6         |
| export_gam_docx . . . . .          | 7         |
| export_knots_csv . . . . .         | 7         |
| extract_smooth_grids . . . . .     | 8         |
| fit_gam . . . . .                  | 9         |
| format_gam_summary . . . . .       | 10        |
| generate_cpp_code . . . . .        | 11        |
| generate_python_code . . . . .     | 11        |
| generate_r_code . . . . .          | 12        |
| glance_gam . . . . .               | 12        |
| import_data . . . . .              | 13        |
| import_earth . . . . .             | 13        |
| mgcvUI . . . . .                   | 14        |
| mod_data_server . . . . .          | 15        |
| mod_data_ui . . . . .              | 15        |
| mod_earth_import_server . . . . .  | 16        |
| mod_earth_import_ui . . . . .      | 16        |
| mod_model_fit_ui . . . . .         | 17        |
| mod_model_results_ui . . . . .     | 17        |
| mod_model_server . . . . .         | 18        |
| mod_report_server . . . . .        | 18        |
| mod_report_ui . . . . .            | 19        |
| mod_variables_params_ui . . . . .  | 19        |
| mod_variables_server . . . . .     | 20        |
| mod_variables_ui . . . . .         | 20        |
| plot_actual_vs_predicted . . . . . | 21        |
| plot_diagnostics . . . . .         | 21        |
| plot_smooths . . . . .             | 22        |
| plot_smooth_interactive . . . . .  | 22        |
| plot_smooth_single . . . . .       | 23        |
| render_gam_report . . . . .        | 24        |
| settings_db . . . . .              | 24        |
| tidy_gam . . . . .                 | 25        |
| <b>Index</b>                       | <b>26</b> |

---

mgcvUI-package

*mgcvUI: Interactive GAM Builder for Real Estate Appraisers*


---

## Description

A Shiny application for building, diagnosing, and reporting Generalized Additive Models (GAMs) via 'mgcv'. Designed for real estate appraisers who need transparent, defensible nonlinear models. Optionally imports knot structure from 'earth' MARS models (e.g. via 'earthUI') to bridge exploratory and confirmatory modelling stages.

## Author(s)

**Maintainer:** mgcvUI Developers <dev@example.com>

**See Also**

Useful links:

- <https://github.com/wcraytor/mgcvUI>
- Report bugs at <https://github.com/wcraytor/mgcvUI/issues>

---

 build\_gam\_formula

*Build a GAM Formula from Variable Specifications*


---

**Description**

Constructs a `stats::formula` suitable for `mgcv::gam()` from a list of smooth-term specifications.

**Usage**

```
build_gam_formula(response, smooth_specs)
```

**Arguments**

- |              |                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| response     | Character scalar – the response variable name.                                                                                                                                                                                                                                                                                                                                                                                                           |
| smooth_specs | A list of smooth-term specification lists. Each element should have: <ul style="list-style-type: none"> <li><b>vars</b> Character vector of variable names (length 1 for univariate smooths, 2+ for tensor products).</li> <li><b>type</b> "s", "te", "ti", or "linear".</li> <li><b>bs</b> Basis type: "tp", "cr", "ps", "bs", etc. Ignored for "linear" and tensor types.</li> <li><b>k</b> Integer basis dimension, or NULL for automatic.</li> </ul> |

**Value**

A `stats::formula` object.

**Examples**

```
specs <- list(
  list(vars = "wt", type = "s", bs = "cr", k = 5L),
  list(vars = "hp", type = "s", bs = "tp", k = NULL),
  list(vars = "cyl", type = "linear", bs = NULL, k = NULL)
)
build_gam_formula("mpg", specs)
```

---

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| build_gam_knots | <i>Build the knots list for gam() from earth knot data</i> |
|-----------------|------------------------------------------------------------|

---

### Description

When earth knots are available for variables using "cr" or "ps" basis types, this function prepares the knots argument for `mgcv::gam()`. If `k` exceeds the number of earth knots (because the GAM should be at least as flexible as standalone), additional knots are placed at data quantiles, preserving the earth-derived positions as anchor points.

### Usage

```
build_gam_knots(smooth_specs, earth_knots, data = NULL)
```

### Arguments

|                           |                                                                                                                                                                                          |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>smooth_specs</code> | List of smooth-term specs (same as <code>build_gam_formula()</code> ).                                                                                                                   |
| <code>earth_knots</code>  | An <code>mgcvUI_earth_knots</code> object, or <code>NULL</code> .                                                                                                                        |
| <code>data</code>         | Data frame used for fitting. Needed to compute quantile-based filler knots when <code>k &gt; length(earth_knots)</code> . If <code>NULL</code> , terms needing augmentation are skipped. |

### Value

A named list suitable for the knots argument of `mgcv::gam()`, or `NULL` if no knots apply.

### Examples

```
specs <- list(
  list(vars = "wt", type = "s", bs = "cr", k = 5L)
)
build_gam_knots(specs, NULL)
```

---

|                        |                                                           |
|------------------------|-----------------------------------------------------------|
| check_sign_consistency | <i>Check for Sign Discrepancies Between Earth and GAM</i> |
|------------------------|-----------------------------------------------------------|

---

### Description

Compares the direction implied by earth hinge functions with the estimated GAM smooth derivatives.

### Usage

```
check_sign_consistency(gam_result)
```

### Arguments

|                         |                                                             |
|-------------------------|-------------------------------------------------------------|
| <code>gam_result</code> | An <code>mgcvUI_result</code> from <code>fit_gam()</code> . |
|-------------------------|-------------------------------------------------------------|

**Value**

A data frame with columns: variable, earth\_direction, gam\_direction, consistent, warning.  
Returns NULL if no earth knots are available.

**Examples**

```
if (requireNamespace("earth", quietly = TRUE)) {
  m <- earth::earth(mpg ~ wt + hp, data = mtcars)
  er <- structure(
    list(model = m, target = "mpg",
          predictors = c("wt", "hp"),
          categoricals = character(0), linpreds = character(0),
          degree = 1L, cv_enabled = FALSE, allowed_matrix = NULL,
          data = mtcars, elapsed = 0, trace_output = character(0)),
    class = "earthUI_result"
  )
  ek <- import_earth(er)
  specs <- list(
    list(vars = "wt", type = "s", bs = "tp", k = NULL),
    list(vars = "hp", type = "s", bs = "tp", k = NULL)
  )
  gam_res <- fit_gam(mtcars, "mpg", specs, earth_knots = ek)
  check_sign_consistency(gam_res)
}
```

---

|                     |                                              |
|---------------------|----------------------------------------------|
| detect_column_types | <i>Detect Column Types from a Data Frame</i> |
|---------------------|----------------------------------------------|

---

**Description**

Returns the R type of each column as a named character vector.

**Usage**

```
detect_column_types(df)
```

**Arguments**

df                      A data frame.

**Value**

Named character vector with values like "numeric", "integer", "factor", "character", "logical", "Date", "POSIXct".

**Examples**

```
detect_column_types(mtcars)
df <- data.frame(
  x = 1:5, y = letters[1:5],
  d = Sys.Date() + 1:5,
  t = Sys.time() + 1:5
)
detect_column_types(df)
```

---

export\_functions\_sqlite

*Export Smooth Functions to SQLite*


---

### Description

Writes smooth function grids to a SQLite database with tables smooth\_grids (variable, x, fx) and model\_info.

### Usage

```
export_functions_sqlite(gam_result, file, n_points = 200L)
```

### Arguments

|            |                                                |
|------------|------------------------------------------------|
| gam_result | An mgcvUI_result from <code>fit_gam()</code> . |
| file       | Character path for the output .sqlite file.    |
| n_points   | Integer grid resolution. Default 200.          |

### Value

The file path (invisibly).

---

export\_functions\_zip

*Export Smooth Functions as a Zip Archive*


---

### Description

Generates code files in selected languages and bundles them into a zip file for download.

### Usage

```
export_functions_zip(
  gam_result,
  file,
  languages = c("R", "Python", "C++", "SQLite"),
  n_points = 200L
)
```

### Arguments

|            |                                                                                    |
|------------|------------------------------------------------------------------------------------|
| gam_result | An mgcvUI_result from <code>fit_gam()</code> .                                     |
| file       | Character path for the output .zip file.                                           |
| languages  | Character vector of languages to include. Options: "R", "Python", "C++", "SQLite". |
| n_points   | Integer grid resolution. Default 200.                                              |

### Value

The zip file path (invisibly).

---

|                 |                                             |
|-----------------|---------------------------------------------|
| export_gam_docx | <i>Export GAM Summary to Word (officer)</i> |
|-----------------|---------------------------------------------|

---

### Description

Creates a Word document with model summary tables using **officer**.

### Usage

```
export_gam_docx(gam_result, file)
```

### Arguments

|            |                                                |
|------------|------------------------------------------------|
| gam_result | An mgcvUI_result from <code>fit_gam()</code> . |
| file       | Character output file path.                    |

### Value

The file path (invisibly).

### Examples

```
if (interactive()) {
  specs <- list(list(vars = "wt", type = "s", bs = "tp", k = NULL))
  res <- fit_gam(mtcars, "mpg", specs)
  export_gam_docx(res, tempfile(fileext = ".docx"))
}
```

---

|                  |                                     |
|------------------|-------------------------------------|
| export_knots_csv | <i>Export Knot Positions to CSV</i> |
|------------------|-------------------------------------|

---

### Description

Writes a tidy CSV with one row per knot, columns: variable, knot, direction.

### Usage

```
export_knots_csv(earth_knots, file)
```

### Arguments

|             |                                                                 |
|-------------|-----------------------------------------------------------------|
| earth_knots | An mgcvUI_earth_knots object from <code>import_earth()</code> . |
| file        | Character path for the output CSV file.                         |

### Value

The file path (invisibly).

**Examples**

```

if (requireNamespace("earth", quietly = TRUE)) {
  m <- earth::earth(mpg ~ wt + hp, data = mtcars)
  er <- structure(
    list(model = m, target = "mpg",
         predictors = c("wt", "hp"),
         categoricals = character(0), linpreds = character(0),
         degree = 1L, cv_enabled = FALSE, allowed_matrix = NULL,
         data = mtcars, elapsed = 0, trace_output = character(0)),
    class = "earthUI_result"
  )
  knots <- import_earth(er)
  tmp <- tempfile(fileext = ".csv")
  export_knots_csv(knots, tmp)
  cat(readLines(tmp), sep = "\n")
  unlink(tmp)
}

```

---

extract\_smooth\_grids    *Extract Smooth Functions as Lookup Tables*

---

**Description**

Evaluates each smooth term on a fine grid and returns x/fx pairs suitable for interpolation in any language.

**Usage**

```
extract_smooth_grids(gam_result, n_points = 200L)
```

**Arguments**

gam\_result        An mgcvUI\_result from `fit_gam()`.

n\_points         Integer number of grid points per variable. Default 200.

**Value**

A named list of data frames, one per smooth variable. Each has columns x and fx.

**Examples**

```

specs <- list(
  list(vars = "wt", type = "s", bs = "tp", k = NULL),
  list(vars = "hp", type = "s", bs = "tp", k = NULL)
)
res <- fit_gam(mtcars, "mpg", specs, cv_folds = 0)
grids <- extract_smooth_grids(res, n_points = 50)
names(grids)

```



fit\_gam

*Fit a GAM Model***Description**

Wrapper around `mgcv::gam()` that accepts the specification objects produced by the mgcvUI interface and returns a structured result.

**Usage**

```
fit_gam(
  data,
  response,
  smooth_specs,
  family = gaussian(),
  method = "REML",
  earth_knots = NULL,
  select = TRUE,
  gamma = 1.2,
  cv_folds = 10L,
  weights = NULL,
  optimizer = NULL,
  scale = 0,
  discrete = FALSE,
  nthreads = 1L
)
```

**Arguments**

|              |                                                                                                                                                           |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| data         | Data frame of observations.                                                                                                                               |
| response     | Character name of the response variable.                                                                                                                  |
| smooth_specs | List of smooth-term specifications (see <code>build_gam_formula()</code> ).                                                                               |
| family       | A family object or character string (e.g. "gaussian", "Gamma"). Default is <code>gaussian()</code> .                                                      |
| method       | Character fitting method. Default "REML".                                                                                                                 |
| earth_knots  | Optional <code>mgcvUI_earth_knots</code> object from <code>import_earth()</code> . When provided, knot positions are passed to <code>mgcv::gam()</code> . |
| select       | Logical. If TRUE, adds an extra penalty to each smooth term so that it can be penalised to zero (variable selection). Default FALSE.                      |
| gamma        | Numeric inflation factor for the effective degrees of freedom in the GCV/UBRE score. Values greater than 1 produce smoother fits. Default 1.              |
| cv_folds     | Integer number of cross-validation folds. Set to 0 to skip CV. Default 10.                                                                                |
| weights      | Optional numeric vector of prior weights (one per observation). Passed to <code>mgcv::gam()</code> .                                                      |
| optimizer    | Character optimizer key: "outer_newton" (default), "outer_bfgs", or "efs".                                                                                |
| scale        | Numeric scale parameter. 0 (default) means estimate; a positive value fixes it.                                                                           |
| discrete     | Logical. If TRUE, uses discretized covariate method for faster fitting on large datasets. Default FALSE.                                                  |
| nthreads     | Integer number of threads for parallel computation. Default 1.                                                                                            |

**Value**

An mgcvUI\_result object (a list) with components:

**model** The fitted `mgcv::gam` object.

**formula** The formula used.

**response** Response variable name.

**smooth\_specs** The smooth-term specs used.

**earth\_knots** The earth knots used (or NULL).

**elapsed** Fitting time in seconds.

**cv\_rsq** Cross-validated R-squared (or NULL if CV skipped).

**Examples**

```
specs <- list(
  list(vars = "wt", type = "s", bs = "tp", k = NULL),
  list(vars = "hp", type = "s", bs = "tp", k = NULL)
)
result <- fit_gam(mtcars, "mpg", specs)
summary(result$model)
```

---

format\_gam\_summary

*Format GAM Summary Statistics*


---

**Description**

Extracts key statistics from a fitted GAM for display in the UI.

**Usage**

```
format_gam_summary(gam_result)
```

**Arguments**

**gam\_result** An mgcvUI\_result from `fit_gam()`.

**Value**

A list with components: `r_squared`, `dev_explained`, `aic`, `bic`, `n_obs`, `n_smooths`, `method`, `family`, `smooth_table`, `parametric_table`.

**Examples**

```
specs <- list(list(vars = "wt", type = "s", bs = "tp", k = NULL))
res <- fit_gam(mtcars, "mpg", specs)
format_gam_summary(res)
```

---

|                   |                                               |
|-------------------|-----------------------------------------------|
| generate_cpp_code | <i>Generate C++ Code for Smooth Functions</i> |
|-------------------|-----------------------------------------------|

---

**Description**

Creates a self-contained C++ header with linear interpolation functions for each smooth term.

**Usage**

```
generate_cpp_code(gam_result, n_points = 200L)
```

**Arguments**

|            |                                                |
|------------|------------------------------------------------|
| gam_result | An mgcvUI_result from <code>fit_gam()</code> . |
| n_points   | Integer grid resolution. Default 200.          |

**Value**

Character string of C++ code.

---

|                      |                                                  |
|----------------------|--------------------------------------------------|
| generate_python_code | <i>Generate Python Code for Smooth Functions</i> |
|----------------------|--------------------------------------------------|

---

**Description**

Creates a self-contained Python script with `numpy.interp()` functions for each smooth term.

**Usage**

```
generate_python_code(gam_result, n_points = 200L)
```

**Arguments**

|            |                                                |
|------------|------------------------------------------------|
| gam_result | An mgcvUI_result from <code>fit_gam()</code> . |
| n_points   | Integer grid resolution. Default 200.          |

**Value**

Character string of Python code.

---

|                 |                                             |
|-----------------|---------------------------------------------|
| generate_r_code | <i>Generate R Code for Smooth Functions</i> |
|-----------------|---------------------------------------------|

---

**Description**

Creates a self-contained R script with `approxfun()` definitions for each smooth term.

**Usage**

```
generate_r_code(gam_result, n_points = 200L)
```

**Arguments**

|                         |                                                             |
|-------------------------|-------------------------------------------------------------|
| <code>gam_result</code> | An <code>mgcvUI_result</code> from <code>fit_gam()</code> . |
| <code>n_points</code>   | Integer grid resolution. Default 200.                       |

**Value**

Character string of R code.

---

|            |                                |
|------------|--------------------------------|
| glance_gam | <i>Glance at GAM Model Fit</i> |
|------------|--------------------------------|

---

**Description**

Returns a one-row data frame of model-level statistics using `broom::glance()`.

**Usage**

```
glance_gam(gam_result)
```

**Arguments**

|                         |                                                             |
|-------------------------|-------------------------------------------------------------|
| <code>gam_result</code> | An <code>mgcvUI_result</code> from <code>fit_gam()</code> . |
|-------------------------|-------------------------------------------------------------|

**Value**

A one-row data frame (tibble).

**Examples**

```
specs <- list(list(vars = "wt", type = "s", bs = "tp", k = NULL))
res <- fit_gam(mtcars, "mpg", specs)
glance_gam(res)
```

---

|             |                                      |
|-------------|--------------------------------------|
| import_data | <i>Import Data from CSV or Excel</i> |
|-------------|--------------------------------------|

---

**Description**

Reads a CSV or Excel file and returns a data frame with cleaned column names (snake\_case, no special characters).

**Usage**

```
import_data(filepath, sheet = 1L, sep = ",", dec = ".")
```

**Arguments**

|          |                                                                                           |
|----------|-------------------------------------------------------------------------------------------|
| filepath | Character path to a .csv, .xlsx, or .xls file.                                            |
| sheet    | Integer or character sheet identifier for Excel files. Ignored for CSV.                   |
| sep      | Character. Field separator for CSV files. Default ",". Use ";" for European-style CSVs.   |
| dec      | Character. Decimal separator for CSV files. Default ".". Use "," for European-style CSVs. |

**Value**

A data frame.

**Examples**

```
tmp <- tempfile(fileext = ".csv")
write.csv(mtcars, tmp, row.names = FALSE)
df <- import_data(tmp)
head(df)
unlink(tmp)
```

---

|              |                                                         |
|--------------|---------------------------------------------------------|
| import_earth | <i>Import an Earth Model and Extract Knot Structure</i> |
|--------------|---------------------------------------------------------|

---

**Description**

Reads a saved earthUI\_result object (.rds file) produced by the **earthUI** package and extracts knot positions, hinge directions, and variable metadata for use in GAM construction.

**Usage**

```
import_earth(path)
```

**Arguments**

|      |                                                                                           |
|------|-------------------------------------------------------------------------------------------|
| path | Character path to an .rds file containing an earthUI_result object, or the object itself. |
|------|-------------------------------------------------------------------------------------------|

**Value**

An mgcvUI\_earth\_knots object (a list) with components:

**knots** Named list of numeric vectors – knot positions per predictor variable.

**signs** Named list of integer vectors – hinge direction at each knot (1 = forward / increasing, -1 = reverse / decreasing).

**interactions** Named list of detected interaction pairs (variable pairs with 2+ active terms in the earth model).

**predictors** Character vector of predictor names used in the earth model.

**target** Character vector of response variable name(s).

**categoricals** Character vector of categorical variable names.

**linpreds** Character vector of variables forced linear in earth.

**degree** Integer max interaction degree from earth (1 = no interactions).

**allowed\_matrix** Matrix of allowed interactions from earthUI (or NULL).

**data** Data frame used to fit the earth model.

**earth\_summary** List of summary statistics from the earth model (R-squared, GCV, number of terms).

**Examples**

```
if (requireNamespace("earth", quietly = TRUE)) {
  m <- earth::earth(mpg ~ wt + hp + disp, data = mtcars)
  er <- structure(
    list(model = m, target = "mpg",
         predictors = c("wt", "hp", "disp"),
         categoricals = character(0), linpreds = character(0),
         degree = 1L, cv_enabled = FALSE, allowed_matrix = NULL,
         data = mtcars, elapsed = 0, trace_output = character(0)),
    class = "earthUI_result"
  )
  knots <- import_earth(er)
  str(knots$knots)
}
```

---

mgcvUI

*Launch the mgcvUI Shiny Application*


---

**Description**

Opens the interactive GAM builder in your default web browser.

**Usage**

```
mgcvUI(port = 7880L, ...)
```

**Arguments**

**port** Integer port number. Defaults to 7880.

**...** Additional arguments passed to [shiny::runApp\(\)](#).

**Value**

Called for its side effect (launches the app). Returns the value of `shiny::runApp()` invisibly.

**Examples**

```
if (interactive()) {
  mgcvUI()
}
```

---

|                 |                                    |
|-----------------|------------------------------------|
| mod_data_server | <i>Data Import Module – Server</i> |
|-----------------|------------------------------------|

---

**Description**

Data Import Module – Server

**Usage**

```
mod_data_server(id)
```

**Arguments**

`id` Shiny module namespace ID.

**Value**

A list with `data` (reactive data frame) and `filename` (reactive original file basename).

---

|             |                                |
|-------------|--------------------------------|
| mod_data_ui | <i>Data Import Module – UI</i> |
|-------------|--------------------------------|

---

**Description**

Compact file upload widget for embedding in a sidebar, with a per-file locale country selector for CSV separator and decimal conventions.

**Usage**

```
mod_data_ui(id)
```

**Arguments**

`id` Shiny module namespace ID.

**Value**

A `shiny::tagList`.

**Examples**

```
if (interactive()) {
  ui <- fluidPage(mod_data_ui("data1"))
  server <- function(input, output, session) {
    mod_data_server("data1")
  }
  shinyApp(ui, server)
}
```

---

mod\_earth\_import\_server

*Earth Import Module – Server*


---

**Description**

Earth Import Module – Server

**Usage**

```
mod_earth_import_server(id)
```

**Arguments**

**id** Shiny module namespace ID.

**Value**

A reactive containing an mgcvUI\_earth\_knots object, or NULL.

---

mod\_earth\_import\_ui     *Earth Import Module – UI*


---

**Description**

Compact widget for importing an earthUI result (.rds file) via file path. Reads directly from disk (no Shiny upload size limits).

**Usage**

```
mod_earth_import_ui(id)
```

**Arguments**

**id** Shiny module namespace ID.

**Value**

A [shiny::tagList](#).



**Examples**

```
if (interactive()) {  
  ui <- fluidPage(mod_earth_import_ui("earth1"))  
  server <- function(input, output, session) {  
    mod_earth_import_server("earth1")  
  }  
  shinyApp(ui, server)  
}
```

---

|                  |                                        |
|------------------|----------------------------------------|
| mod_model_fit_ui | <i>Model Fit Button – UI (sidebar)</i> |
|------------------|----------------------------------------|

---

**Description**

Model Fit Button – UI (sidebar)

**Usage**

```
mod_model_fit_ui(id)
```

**Arguments**

`id` Shiny module namespace ID.

**Value**

A [shiny::tagList](#).

**Examples**

```
if (interactive()) {  
  ui <- fluidPage(mod_model_fit_ui("model1"))  
}
```

---

|                      |                                        |
|----------------------|----------------------------------------|
| mod_model_results_ui | <i>Model Results – UI (main panel)</i> |
|----------------------|----------------------------------------|

---

**Description**

Tabbed output panel for model results, plots, and diagnostics.

**Usage**

```
mod_model_results_ui(id)
```

**Arguments**

`id` Shiny module namespace ID.

**Value**

A [shiny::tagList](#).

---

|                  |                                                                 |
|------------------|-----------------------------------------------------------------|
| mod_model_server | <i>Model Module – Server (shared by fit button and results)</i> |
|------------------|-----------------------------------------------------------------|

---

### Description

Model Module – Server (shared by fit button and results)

### Usage

```
mod_model_server(
  id,
  data_r,
  var_config_r,
  earth_knots_r = reactive(NULL),
  dark_mode_r = reactive(FALSE)
)
```

### Arguments

|               |                                                     |
|---------------|-----------------------------------------------------|
| id            | Shiny module namespace ID.                          |
| data_r        | A reactive returning the data frame.                |
| var_config_r  | A reactive returning the variable configuration.    |
| earth_knots_r | A reactive returning earth knots (or NULL).         |
| dark_mode_r   | A reactive returning TRUE when dark mode is active. |

### Value

A reactive containing the `mgcvUI_result`, or NULL.

---

|                   |                                      |
|-------------------|--------------------------------------|
| mod_report_server | <i>Report Export Module – Server</i> |
|-------------------|--------------------------------------|

---

### Description

Report Export Module – Server

### Usage

```
mod_report_server(id, gam_result_r, data_r)
```

### Arguments

|              |                                                      |
|--------------|------------------------------------------------------|
| id           | Shiny module namespace ID.                           |
| gam_result_r | A reactive returning an <code>mgcvUI_result</code> . |
| data_r       | A reactive returning the data frame.                 |

---

`mod_report_ui`*Report Export Module – UI*

---

**Description**

Compact download controls for sidebar embedding.

**Usage**

```
mod_report_ui(id)
```

**Arguments**

`id` Shiny module namespace ID.

**Value**

A [shiny::tagList](#).

**Examples**

```
if (interactive()) {  
  ui <- fluidPage(mod_report_ui("report1"))  
}
```

---

`mod_variables_params_ui`*Mgcv Call Parameters – UI*

---

**Description**

Model fitting parameters for the GAM call. Uses the same module namespace as [mod\\_variables\\_ui\(\)](#) so inputs are shared.

**Usage**

```
mod_variables_params_ui(id)
```

**Arguments**

`id` Shiny module namespace ID (must match [mod\\_variables\\_ui\(\)](#)).

**Value**

A [shiny::tagList](#).

---

|                      |                                           |
|----------------------|-------------------------------------------|
| mod_variables_server | <i>Variable Selection Module – Server</i> |
|----------------------|-------------------------------------------|

---

### Description

Variable Selection Module – Server

### Usage

```
mod_variables_server(
  id,
  data_r,
  filename_r = reactive(NULL),
  earth_knots_r = reactive(NULL),
  purpose_r = reactive("general")
)
```

### Arguments

|               |                                                                                     |
|---------------|-------------------------------------------------------------------------------------|
| id            | Shiny module namespace ID.                                                          |
| data_r        | A reactive returning the imported data frame.                                       |
| filename_r    | A reactive returning the original filename.                                         |
| earth_knots_r | A reactive returning an mgcvUI_earth_knots object (or NULL).                        |
| purpose_r     | A reactive returning the purpose mode string ("general", "appraisal", or "market"). |

### Value

A reactive list with components: response, predictors, smooth\_specs, family, method, select, gamma.

---

|                  |                                       |
|------------------|---------------------------------------|
| mod_variables_ui | <i>Variable Selection Module – UI</i> |
|------------------|---------------------------------------|

---

### Description

Compact variable table with Inc, Type, and Linear checkboxes, plus an Allowed Interactions matrix. Designed for sidebar embedding.

### Usage

```
mod_variables_ui(id)
```

### Arguments

|    |                            |
|----|----------------------------|
| id | Shiny module namespace ID. |
|----|----------------------------|

### Value

A [shiny::tagList](#).

**Examples**

```
if (interactive()) {
  ui <- fluidPage(mod_variables_ui("vars1"))
}
```

---

```
plot_actual_vs_predicted
```

*Plot Actual vs Predicted*

---

**Description**

Plot Actual vs Predicted

**Usage**

```
plot_actual_vs_predicted(gam_result)
```

**Arguments**

gam\_result      An mgcvUI\_result from `fit_gam()`.

**Value**

A ggplot object.

**Examples**

```
specs <- list(list(vars = "wt", type = "s", bs = "tp", k = NULL))
res <- fit_gam(mtcars, "mpg", specs)
plot_actual_vs_predicted(res)
```

---

```
plot_diagnostics
```

*Plot GAM Residual Diagnostics*

---

**Description**

Produces a 2x2 panel of diagnostic plots: residuals vs fitted, Q-Q plot, histogram of residuals, and response vs fitted.

**Usage**

```
plot_diagnostics(gam_result)
```

**Arguments**

gam\_result      An mgcvUI\_result from `fit_gam()`.

**Value**

A ggplot object (patchwork-assembled if available, otherwise individual plots returned as a list).

**Examples**

```
specs <- list(list(vars = "wt", type = "s", bs = "tp", k = NULL))
res <- fit_gam(mtcars, "mpg", specs)
plot_diagnostics(res)
```

---

plot\_smooths

*Plot GAM Smooth Terms*


---

**Description**

Produces ggplot2-based smooth plots for each term in the fitted GAM, using the **gratia** package for tidy extraction.

**Usage**

```
plot_smooths(gam_result, residuals = TRUE)
```

**Arguments**

gam\_result      An mgcvUI\_result from `fit_gam()`.  
 residuals      Logical. If TRUE, overlay partial residuals on smooth plots. Default TRUE.

**Value**

A ggplot object showing all smooth terms in a faceted layout.

**Examples**

```
specs <- list(
  list(vars = "wt", type = "s", bs = "tp", k = NULL),
  list(vars = "hp", type = "s", bs = "tp", k = NULL)
)
res <- fit_gam(mtcars, "mpg", specs)
plot_smooths(res)
```

---

plot\_smooth\_interactive

*Interactive Smooth Plot with Slope Tooltip*


---

**Description**

Creates a plotly smooth plot showing contribution to value with hover tooltips displaying the slope (rate of change per unit). For latitude/longitude variables, slope is computed per 0.001 degrees.

**Usage**

```
plot_smooth_interactive(
  gam_result,
  variable,
  earth_knots = NULL,
  dark_mode = FALSE
)
```

**Arguments**

|             |                                                 |
|-------------|-------------------------------------------------|
| gam_result  | An mgcvUI_result from <code>fit_gam()</code> .  |
| variable    | Character name of the variable to plot.         |
| earth_knots | Optional mgcvUI_earth_knots object.             |
| dark_mode   | Logical; use dark theme styling. Default FALSE. |

**Value**

A plotly htmlwidget.

---

|                    |                                      |
|--------------------|--------------------------------------|
| plot_smooth_single | <i>Plot a Single GAM Smooth Term</i> |
|--------------------|--------------------------------------|

---

**Description**

Plots one smooth term from the fitted GAM with confidence band and optional partial residuals.

**Usage**

```
plot_smooth_single(gam_result, variable, residuals = TRUE, earth_knots = NULL)
```

**Arguments**

|             |                                                                                                                        |
|-------------|------------------------------------------------------------------------------------------------------------------------|
| gam_result  | An mgcvUI_result from <code>fit_gam()</code> .                                                                         |
| variable    | Character name of the variable to plot.                                                                                |
| residuals   | Logical. Overlay partial residuals? Default TRUE.                                                                      |
| earth_knots | Optional mgcvUI_earth_knots object. If provided, earth knot positions are marked on the plot as vertical dashed lines. |

**Value**

A ggplot object.

**Examples**

```
specs <- list(
  list(vars = "wt", type = "s", bs = "tp", k = NULL),
  list(vars = "hp", type = "s", bs = "tp", k = NULL)
)
res <- fit_gam(mtcars, "mpg", specs)
plot_smooth_single(res, "wt")
```

---

|                   |                            |
|-------------------|----------------------------|
| render_gam_report | <i>Render a GAM Report</i> |
|-------------------|----------------------------|

---

### Description

Generates an HTML, PDF, or Word report for a fitted GAM model.

### Usage

```
render_gam_report(
  gam_result,
  output_format = "html",
  output_file = NULL,
  title = "GAM Model Report"
)
```

### Arguments

|               |                                                                                     |
|---------------|-------------------------------------------------------------------------------------|
| gam_result    | An mgcvUI_result from <code>fit_gam()</code> .                                      |
| output_format | Character: "html", "pdf", or "docx". Default "html".                                |
| output_file   | Character path for the output file. If NULL (default), a temporary file is created. |
| title         | Character report title. Default "GAM Model Report".                                 |

### Value

The output file path (invisibly).

### Examples

```
if (interactive()) {
  specs <- list(list(vars = "wt", type = "s", bs = "tp", k = NULL))
  res <- fit_gam(mtcars, "mpg", specs)
  render_gam_report(res, "html")
}
```

---

|             |                                                                |
|-------------|----------------------------------------------------------------|
| settings_db | <i>Settings Persistence (SQLite, keyed by input file name)</i> |
|-------------|----------------------------------------------------------------|

---

### Description

Stores and retrieves per-file variable configuration so that settings are remembered when the same data file is re-opened.



---

tidy\_gam

---

*Tidy GAM Results with broom***Description**

Returns a tidy data frame of model terms using `broom::tidy()`.

**Usage**

```
tidy_gam(gam_result)
```

**Arguments**

`gam_result`      An mgcvUI\_result from `fit_gam()`.

**Value**

A data frame (tibble) of tidied model terms.

**Examples**

```
specs <- list(list(vars = "wt", type = "s", bs = "tp", k = NULL))
res <- fit_gam(mtcars, "mpg", specs)
tidy_gam(res)
```

# Index

## \* internal

- mgcvUI-package, 2
- settings\_db, 24
  
- broom::glance(), 12
- broom::tidy(), 25
- build\_gam\_formula, 3
- build\_gam\_formula(), 4, 9
- build\_gam\_knots, 4
  
- check\_sign\_consistency, 4
  
- detect\_column\_types, 5
  
- export\_functions\_sqlite, 6
- export\_functions\_zip, 6
- export\_gam\_docx, 7
- export\_knots\_csv, 7
- extract\_smooth\_grids, 8
  
- fit\_gam, 9
- fit\_gam(), 4, 6–8, 10–12, 21–25
- format\_gam\_summary, 10
  
- generate\_cpp\_code, 11
- generate\_python\_code, 11
- generate\_r\_code, 12
- glance\_gam, 12
  
- import\_data, 13
- import\_earth, 13
- import\_earth(), 7, 9
  
- mgcv::gam, 10
- mgcv::gam(), 3, 4, 9
- mgcvUI, 14
- mgcvUI-package, 2
- mod\_data\_server, 15
- mod\_data\_ui, 15
- mod\_earth\_import\_server, 16
- mod\_earth\_import\_ui, 16
- mod\_model\_fit\_ui, 17
- mod\_model\_results\_ui, 17
- mod\_model\_server, 18
- mod\_report\_server, 18
- mod\_report\_ui, 19
- mod\_variables\_params\_ui, 19
- mod\_variables\_server, 20
- mod\_variables\_ui, 20
- mod\_variables\_ui(), 19
  
- plot\_actual\_vs\_predicted, 21
- plot\_diagnostics, 21
- plot\_smooth\_interactive, 22
- plot\_smooth\_single, 23
- plot\_smooths, 22
  
- render\_gam\_report, 24
  
- settings\_db, 24
- shiny::runApp(), 14, 15
- shiny::tagList, 15–17, 19, 20
- stats::formula, 3
  
- tidy\_gam, 25